

Eradication Phase

4/15/26 1:56 PM

SUMMARY:

Purpose: Completely remove attacker access and prevent reinfection.

When: After full scoping from host + network analysis.

Core Rule: Blocking alone is not enough — you must identify every infected system and all persistence.

Key Steps:

- 1. Scope infected hosts** — Use **tcpdump** on PCAP to extract all internal IPs talking to C2.
- 2. Detect web shells** — `nmap -sS -Pn -p 8080 <subnet>`.
- 3. Find malware copies** — strings for unique strings, **YARA** rules for content-based scanning, **Velociraptor** for scale.
- 4. Block** — Firewall/ACL rules for C2 IPs and malicious domains.
- 5. Verify** — Confirm no remaining persistence (tasks, services, scripts).

Main Tools: tcpdump, nmap, strings, YARA, Velociraptor, firewalls/ACLs.

Why it matters: Incomplete eradication leads to re-infection. Network-level blocking cuts off C2 and breaks daisy-chained implants.

When to perform: After full scoping from host + network analysis. **Why:** Remove attacker access and all malware/persistence to prevent reinfection. Simply deleting files is insufficient — you must block C2 and identify every copy.

Core goal: Block attacker, scope every infected system, remove all persistence and malware instances.

Investigation Workflow (Step-by-Step Order)

- Final scoping (find all infected hosts).
- Detect web shells and persistence.
- Locate all malware copies using signatures.
- Block C2 and malicious domains.
- Verify and prepare for recovery.

1. Network Scoping (Find Infected Hosts)

- Tool: tcpdump** (on captured PCAP) Full pipeline:
Bash
`tcpdump -nr ext_traffic.pcap host <C2_IP> | cut -d " " -f3 | cut -d "." -f1-4 | sort -n | uniq`
Why: Extracts all internal source IPs that talked to the C2 server. These are compromised hosts.

2. Web Shell Detection

- Tool: nmap**
Bash
`sudo nmap -sS -Pn -p 8080 <subnet>`
Then: `cat scan.gnmap | grep open` **Why:** Finds open ports (e.g., 8080) where web shells provide persistent attacker access. -sS = SYN scan (fast), -Pn = skip ping.

3. Malware File Scoping

- Tool: strings**
Bash
`strings malware.exe | more`
or with length filter: `strings -n 10 malware.exe` **Why:** Extracts readable strings (compiler info, unique phrases, ransom notes) to create signatures. File names can be changed, but content cannot.
- Tool: YARA** Create rule with strings (e.g., "Go build ID", unique phrases), then scan. **Why:** Detects malware by content across the filesystem, even if renamed.
- Tool: Velociraptor** (with YARA artifact, e.g., `Windows.Detection.Yara.NTFs`) **Why:** Scales scanning to all endpoints at once.

4. Blocking

- Tool:** Firewall / Network ACLs (or AWS Security Groups + NACLs) Block C2 IP (e.g., port 443) and malicious domain IP (use nslookup first). **Why:** Cuts off attacker control. Rule order matters — DENY rules before broad ALLOW. In AWS, NACLs support explicit DENY.

Effects: Attacker loses C2, implants fail, daisy-chained hosts lose connectivity. Some malware (including certain ransomware) may stop functioning without internet.

Remaining threats: Automated persistence (tasks, services) that do not require live C2.

Eradication Checklist:

- C2 IP and malicious domains blocked.
- All infected hosts identified.
- All malware files and web shells located/removed.
- All persistence mechanisms eliminated.

Tools summary: tcpdump (scoping), nmap (ports), strings (signatures), YARA (detection), Velociraptor (scale), nslookup (domains), firewalls/ACLs (blocking).

Key takeaway: Eradication requires complete scoping + network-level blocking before recovery.

Windows Event Logs, Analysis, Registry & File System Forensics

4/15/26 1:57 PM

When to perform: During/after host analysis for deeper reconstruction, timelines, and persistence details. **Why:** These artifacts survive reboots and provide reliable evidence of authentication, execution, file activity, and tampering.

Major Artifacts & What They Reveal

Artifact	Reveals	Common Tools
Windows Event Logs	Logons, process creation, RDP, privilege use	Event Viewer, EvtxECmd, Chainsaw
UserAssist / RecentDocs	GUI program execution and recently opened files	Registry Explorer, RECmd
ShellBags	Folder access (including deleted/USB/network)	ShellBags Explorer, SBECmd
ShimCache (AppCompatCache)	Files that existed/run on the system	AppCompatCacheParser
AmCache	Executed programs with paths and hashes	AmcacheParser
\$MFT (Master File Table)	Full filesystem timeline (create/modify/access/delete)	MFTECmd, MFTEExplorer, Autopsy
Browser History	Web activity, downloads, attacker research	Hindsight, BrowsingHistoryView
Registry (Run keys, Services, Tasks)	Persistence	Autoruns, RECmd, Registry Explorer

Windows Event Log Analysis

- **Tool: Windows Event Viewer** (or export tools) Key events:
 - **4624** (successful logon — note logon type: 3=network, 10=RDP)
 - **4625** (failed logon)
 - **4688** (process creation — enable command-line logging for details) **Why:** Shows who logged in, when, and what processes ran. Convert times to UTC and filter noise.

NTFS File System Forensics

- **Tool: MFTECmd** or **MFTEExplorer** Example: MFTECmd.exe -f \$MFT --csv timeline.csv **Why:** \$MFT is the central filesystem record. Every file has an entry with timestamps. Use for timelines (filter .exe, Temp folder, etc.).
Timestamps (MACB): M=Modified, A=Accessed, C=Changed (metadata), B=Born (creation). Compare SI (easily altered) vs FN (more reliable) entries.
Small files (< ~700 bytes) can be resident (content stored in MFT — recoverable even if deleted). Additional: \$LogFile and \$UsnJrnl track changes.

Registry & Execution Artifacts

- **Tool: Registry Explorer** or **RECmd** Check UserAssist, RecentDocs, Run keys, etc. **Why:** Reveals GUI executions, opened files, and persistence locations.

Browser Forensics

- **Tool: BrowsingHistoryView** or **Hindsight** **Why:** Aggregates history/downloads across browsers. Shows intent (e.g., research on tools or exfil sites).

Overall Investigation Flow

1. Event logs → authentication and process activity.
2. NTFS/MFT → what changed on disk and when (timelines).
3. Registry artifacts → execution history and persistence.
4. Browser → external activity and intent.
5. Correlate everything into one timeline.

Detection & Analysis / Initial Triage

4/15/26 1:39 PM

SUMMARY:

Purpose: Quickly scope an incident while the system is still live. Collect volatile evidence before it disappears on reboot.

When: Right after an alert or suspicion of compromise.

Core Rule: Collect first. Analyze later.

Key Steps (in order):

1. **Documentation** — Start transcript and captains log (audit trail).
2. **System Info** — systeminfo or Get-ComputerInfo (OS, patches, uptime).
3. **Processes** — tasklist + **Process Explorer** (parent-child, suspicious parents, injection).
4. **Network** — netstat -ano (connections + PID), ipconfig /displaydns, arp -a.
5. **Users & Accounts** — net user, psloggedon, logonsessions.
6. **Persistence** — **Autoruns**, schtasks /query, services.
7. **Logs & Traffic** — Copy EVTX logs, capture with **Wireshark** or **RawCap**.

Main Tools: Process Explorer, Autoruns, netstat, Wireshark. **Why it matters:** Volatile data (processes, connections, memory) is lost on reboot. This phase decides if deeper host/network analysis is needed.

1. Detection & Analysis / Initial Triage

When to perform: Right after receiving an alert or suspecting compromise, while the system is still live (before reboot or shutdown). **Why:** Volatile data (processes, network connections, memory) disappears on reboot. This phase quickly identifies signs of compromise, captures evidence, determines scope, and decides if deeper analysis is needed. **Core rule:** **Collect evidence first. Analyze later.** Save all artifacts to a dedicated folder (\$datapath).

Investigation Workflow (Step-by-Step Order)

1. Start documentation (audit trail).
2. Gather basic system information (context).
3. Investigate running processes (malware runs as processes).
4. Check network connections and related artifacts (C2 communication).
5. Examine users, accounts, and logged-in sessions (unauthorized access).
6. Check services and persistence mechanisms (survival after reboot).
7. Collect logs and additional artifacts.
8. Capture network traffic if suspicious connections are found.

1. Documentation (PowerShell)

- **Tool:** PowerShell
PowerShell
Start-Transcript -Path \$datapath\transcript.txt
Why: Records every command you run and its output. Creates a defensible forensic audit trail of your actions.
- **Tool:** PowerShell
PowerShell
echo "captains_log: starting triage \$(Get-Date)" >> \$datapath\captains_log.txt
Why: Adds timestamped investigator notes for important observations.

2. Basic System Information (Built-in Windows / PowerShell)

- **Tool:** Command Prompt or PowerShell
cmd
systeminfo
or
PowerShell
Get-ComputerInfo
Why: Shows OS version, patch level, hostname, boot time, hardware, and domain membership. Reveals outdated systems, long uptime (possible compromise), or unusual configurations.

3. Running Processes Investigation

Processes are the most critical volatile artifact because malware must execute as a process.

- **Tool:** Built-in Windows (tasklist)
cmd
tasklist
Why: Lists processes, PIDs, and memory usage. Quickly spot strange names, high CPU/memory usage, or unknown executables.
- **Tool:** Sysinternals **Process Explorer** (procexp.exe) Run procexp.exe (GUI tool). **Why:** Displays parent-child relationships, loaded DLLs, network connections per process, and digital signatures. Detects process injection, suspicious parents (e.g., malware spawned from explorer.exe), or unsigned binaries.

4. Network Connection Investigation

Attackers need to communicate with C2 servers or other compromised systems.

- **Tool:** Built-in Windows (netstat)
cmd
netstat -ano
Why: Shows active connections, remote IPs/ports, states, and owning PID. Identify outbound C2 traffic or unusual internal connections.
Save output:
cmd
netstat -ano > \$datapath\netconn.txt
- **Tool:** Built-in Windows (ipconfig)
cmd
ipconfig /displaydns
Why: Displays recent DNS cache. Reveals malware domains, C2 servers, or suspicious lookups.
- **Tool:** Built-in Windows (arp)
cmd
arp -a
Why: Shows local devices recently communicated with. Helps detect lateral movement on the LAN.
- **Tool:** PowerShell
PowerShell
Get-NetRoute
Why: Shows routing table. Malware may manipulate routes to redirect traffic.
- **Tool:** PowerShell
PowerShell
Get-NetFirewallRule
Why: Lists firewall rules. Attackers often add rules to allow their traffic or bypass security.

5. User & Account Investigation

Attackers create or use accounts for persistence and privilege escalation.

- **Tool:** Built-in Windows (net)
cmd
net user
and
cmd
net localgroup administrators
Why: Lists accounts and admin group members. Spot new or suspicious users.
- **Tool:** Sysinternals (psloggedon.exe) Run psloggedon.exe. **Why:** Shows currently logged-on users (local and remote).
- **Tool:** Sysinternals (logonsessions64.exe) Run logonsessions64.exe. **Why:** Details session IDs, login times, and authentication types. Useful for correlating with event logs later.
- **Tool:** Built-in Windows (net)
cmd
net use
Why: Shows mapped network shares/drives. Ransomware often spreads via shares.

6. Services & Persistence Investigation

Malware installs as services or scheduled tasks to survive reboots.

- **Tool:** Built-in Windows (sc)
cmd
sc query
or PowerShell:
PowerShell
Get-WmiObject Win32_Service
Why: Lists services. Look for suspicious names or unknown binaries.
- **Tool:** Sysinternals **Autoruns** (autoruns.exe) Run autoruns.exe (GUI). **Why:** Scans startup items, scheduled tasks, services, drivers, and registry run keys. Finds persistence mechanisms.
- **Tool:** Built-in Windows (schtasks)
cmd
schtasks /query /fo LIST /v
Why: Lists scheduled tasks. Attackers use them to run malware or reconnect to C2.
- **Tool:** Sysinternals (listdlls.exe) Run listdlls.exe. **Why:** Shows loaded DLLs per process. Detects DLL injection or malicious libraries.
- **Tool:** Sysinternals (ntfsinfo.exe)
cmd
ntfsinfo C:
Why: Provides NTFS filesystem details. Useful for later timeline creation.

7. Log Collection & Traffic Capture

- **Tool:** PowerShell (copy)
PowerShell
Copy-Item C:\Windows\System32\winevt\Logs* \$datapath\winevent_logs
Why: Preserves raw EVTX logs (logins, process creation, etc.) for offline analysis.
- **Tool:** PowerShell (copy)
PowerShell
Copy-Item C:\Windows\System32\LogFiles* \$datapath\LogFiles
Why: Captures additional logs (firewall, IIS, etc.).
- **Tool:** **Wireshark** (GUI) Use filters such as: tcp.port == 8080 or ip.addr == <suspect IP> **Why:** Inspects live or captured traffic. Detects web shells or specific attacker commands.
- **Tool:** **RawCap** (rawcap.exe) — portable alternative when Wireshark cannot be installed. Run rawcap.exe to list interfaces, then capture. **Why:** Allows quick packet capture without installation.

Goal of phase: Quickly scope the incident, collect volatile evidence, and decide next steps (deeper host or network analysis).

Host Analysis

4/15/26 1:55 PM

SUMMARY:

- Purpose:** Reconstruct what the attacker did on a compromised endpoint.
- When:** After triage confirms compromise on a specific machine.
- Core Approach:** Follow the chain — Connection → Process → Memory → Payload → File → Logs → Timeline → Root Cause.
- Key Steps:**
1. Recap triage (match netstat PIDs to processes).
 2. **Memory analysis** (fileless threats) — **Volatility** (pslist, pstree, cmdline, netscan).
 3. Decode payloads — **CyberChef** (Base64, UTF-16).
 4. Analyze initial access files — **oletools** (oleid, olevba for macros).
 5. Review logs — Event ID **4624** (logon), **4688** (process creation).
 6. Build timeline & check persistence — **Velociraptor** (MFT timeline), Autoruns.
 7. Correlate everything into attack story.

Main Tools: Volatility, CyberChef, oletools, Velociraptor, Event Viewer.

Why it matters: Reveals initial access, execution, persistence, and impact on the host.

When to perform: After initial triage confirms compromise on an endpoint.

Why: Reconstruct attacker actions on the machine — initial access, execution, persistence, impact, and root cause. Follow the evidence chain: Network connection → Process → Memory → Payload → File → Logs → Timeline.

- Key Tools & Purpose:**
- **Velociraptor:** Remote DFIR collection, queries, and timelines across endpoints.
 - **Volatility:** Memory forensics (especially for fileless malware that leaves little disk footprint).
 - **CyberChef:** Decode obfuscated/encoded payloads (Base64, UTF-16, etc.).
 - **oletools** (oleid, olevba): Analyze malicious Office documents and extract macros.
 - **Windows Event Viewer:** Review built-in logs for logins, processes, RDP, etc.

Investigation Workflow (Step-by-Step Order)

1. Quick triage recap (network + processes).
2. Memory analysis (fileless threats).
3. Decode payloads.
4. Analyze initial access files (e.g., documents).
5. Review logs.
6. Build timelines and check persistence.
7. Correlate findings to understand full attacker activity.

1. Quick Triage Recap

- **Tool:** Built-in Windows cmd
`netstat -ano`
and
cmd
`tasklist`
Why: Match suspicious network connections (PIDs) to running processes. Identifies what is communicating with attackers or other systems.

2. Memory Analysis (for fileless attacks)

- **Tool:** **Volatility** (run on a memory dump) Common commands:
Bash
`volatility -f memory.raw pslist`
or
Bash
`volatility -f memory.raw pstree`
Why: Lists processes and parent-child trees. Reveals hidden or injected processes that may not appear in live tasklist.
Bash
`volatility -f memory.raw cmdline`
Why: Shows full command-line arguments. Often reveals encoded PowerShell or suspicious parameters.
Bash
`volatility -f memory.raw netscan`
Why: Extracts network connections from memory. Useful when live netstat is insufficient.

3. Decode Obfuscated Payloads

- **Tool:** **CyberChef** (web or desktop) Recipes: Base64 decode → UTF-16LE decode. Or PowerShell one-liner:
PowerShell
`[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String("ENCODED_STRING"))`
Why: Attackers heavily obfuscate payloads. Decoding reveals the real commands or next-stage malware.

4. File & Initial Access Analysis

- **Tool:** **oletools**
Bash
`oleid suspicious.doc`
Why: Detects macros or other malicious features in Office files.
Bash
`olevba suspicious.doc`
Why: Extracts and displays macro code (e.g., AutoOpen that runs PowerShell).

5. Log Analysis

- **Tool:** **Windows Event Viewer** (or tools like EvtxECmd/Chainsaw)
Focus on Security log:
 - Event ID **4624** (successful logon)
 - **4625** (failed logon)
 - **4688** (process creation) **Why:** Reveals logins (including RDP type 10), process executions, and privilege use. Correlate with triage findings.

6. Timeline & Persistence

- **Tool:** **Velociraptor** Use artifacts like Windows.Timeline.MFT or YARA scans. **Why:** Builds timelines of file activity, new executables, or downloads. Helps reconstruct sequence of events.
- **Tool:** **Autoruns** or registry checks (from triage) **Why:** Confirms persistence (run keys, services, tasks).

Mindset: Always follow the chain of evidence and correlate across tools to build a clear attack story on the host.

Network Analysis

4/15/26 1:56 PM

SUMMARY:

Purpose: See the full attack picture across systems (C2, lateral movement, exfiltration).

When: After host analysis shows suspicious traffic or when scoping multiple systems.

Core Approach: Identify who talked to whom, how often, and why.

Key Steps:

1. Spot suspicious communication (unknown domains, high volume, strange ports).
2. Find infected hosts — filter by source IP in **Kibana**.
3. Track lateral movement (SMB port 445).
4. Investigate C2/beaconing (regular connections to same IP/port).
5. Detect scanning (many SYN without handshake).
6. Check exfiltration (DNS tunneling — high-entropy queries).
7. Extract IOCs (IPs, domains, techniques).

Main Tools: **Kibana** (Elasticsearch), **Zeek** (structured logs from PCAP), **tcpdump**. **Why it matters:** Reveals east-west traffic and the complete timeline that individual hosts miss.

When to perform: After host analysis reveals suspicious traffic, or when you need visibility across the environment. **Why:** Shows the full attack picture — C2, lateral movement, scanning, exfiltration — that individual hosts may miss. Builds an end-to-end timeline.

Main Tools:

- **Kibana** (with Elasticsearch): Fast log searching and filtering.
- **Zeek**: Turns PCAPs into structured logs (conn.log, http.log, dns.log, etc.).
- **tcpdump**: Raw packet capture and inspection.

Investigation Workflow (Step-by-Step Order)

1. Identify suspicious communication.
2. Find infected host(s).
3. Track lateral movement.
4. Investigate C2 and beaconing.
5. Detect scanning/enumeration.
6. Check critical servers and exfiltration.
7. Extract IOCs.

1. Identify Suspicious Communication

- **Tool:** **Kibana** Filter by unknown domains, strange ports, or high volume. **Why:** Spots anomalies like outbound traffic to rare domains.

2. Identify Infected Host

- **Tool:** **Kibana** Filter: source.ip == <suspect IP> **Why:** Narrows down which internal machine is communicating externally.

3. Track Lateral Movement

- **Tool:** **tcpdump** or **Kibana** (port 445/SMB) Example:
Bash
`tcpdump -nn -r incident.pcap host <IP>`
Why: Reveals internal SMB traffic used for file sharing or remote execution (common in ransomware spread).

4. Investigate C2 & Beaconing

- **Tool:** **Kibana** Filter for repeated connections to same IP/port (often 443). **Why:** Beaconing = malware calling home at regular intervals with consistent packet sizes.
- **Tool:** **Zeek** (cat conn.log or http.log) **Why:** Provides structured data on connections and downloads.

5. Detect Scanning

- **Tool:** **Kibana** or **tcpdump** Look for many SYN packets without full handshake (SYN scan). **Why:** Attackers scan for open ports or vulnerable systems.

6. Investigate Exfiltration & Critical Servers

- **Tool:** **Zeek** (dns.log + entropy scripts) High-entropy DNS queries indicate tunneling. **Why:** DNS can exfiltrate data covertly (high volume or encoded subdomains).

Mindset: Ask — Who talked to whom? How often? Why that port? Combine with host data for complete visibility. Always examine internal (east-west) traffic.

Goal: Build full attack timeline and identify all compromised assets + IOCs (IPs, domains, techniques).

GRC + Incident + Pentesting

4/24/26 8:56 PM

1. Core Mindset (Foundation)

- **Goal:** Manage risk so attacks are not worth it (cost > reward).
- **Think like an attacker:** balance **effort, cost, reward**.
- **Security exists to support business**, not block it.

CIA Triad (always baseline)

- **Confidentiality** – prevent unauthorized access
- **Integrity** – prevent unauthorized changes
- **Availability** – ensure access when needed

2. Risk Management (Core Engine)

Risk Formula

- **Risk = Impact × Likelihood**
- **Residual Risk:** what remains after controls → compare to **risk appetite**

Impact Categories (always assess together)

1. Safety (highest)
2. Financial
3. Reputational
4. Regulatory/Legal

Likelihood (based on controls)

- No controls → **Almost Certain**
- Partial → **Probable**
- Strong/layered → **Possible** (never zero)

Defense in Depth (3 Layers)

1. **Endpoints / Entry** – MFA, patching
2. **Network Boundaries** – firewalls, segmentation
3. **End-to-End** – encryption, monitoring

Attack Surface Drivers

- Size
- Exposure (internet-facing)
- Complexity / distribution

3. Business Impact Analysis (BIA)

Purpose

- Answer: **“What happens if this breaks?”**

Key Metrics

- **RTO:** max downtime allowed
- **RPO:** max data loss allowed

Approach

- Map: **Infrastructure → Apps → Business Services**
- Prioritize **critical business functions**
- Consider **peak vs off-peak impact**

Key Insight

- Supporting systems must often have **stricter RTO/RPO** than the business service.

4. Governance (Structure & Control)

Governance vs Management

- **Governance:** direction, oversight (board/CISO)
- **Management:** execution (IT/security teams)

Core Components

- **Policies** → what/why
 - **Standards** → specific requirements
 - **Procedures** → step-by-step
 - **Processes** → end-to-end workflows
- 👉 If these don't align → governance fails.

Scope & SoA

- **Scope:** what systems/data are included
- **SoA (Statement of Applicability):**
 - Which controls apply (e.g., ISO 27001)
 - Why + implementation status

Controls (3 Types)

- **Preventive:** stop attacks (MFA)
- **Detective:** find attacks (logs)
- **Corrective:** fix issues (backups)

Key Governance Concepts

- **Separation of Duties:** no single person controls everything
- **Exception Handling:** controlled policy bypass
- **Maturity Models (CMMI):** improve over time

Compliance Types

- **Statutory:** laws (GDPR, HIPAA)
- **Regulatory:** industry (PCI-DSS)
- **Contractual:** agreements
- **Voluntary:** ISO, certifications

Operating Model

- Work across:
 - HR (onboarding/offboarding)
 - Legal (compliance)
 - Procurement (vendors)

👉 You act as the **bridge between departments**

5. Risk Across Lifecycle

Phases

1. **Discovery:** identify inherent risk
2. **Design:** architecture risks
3. **Build/Test:** validation, pentesting
4. **Deploy:** readiness + residual risk
5. **Operate:** monitoring, incidents
6. **Decommission:** secure disposal

Assessment Approaches

- **Top-down:** business impact
- **Bottom-up:** technical controls
- ☒ Best = **combine both**

Key Improvement Concepts

- Avoid bias/overconfidence
- Move unknown risks → known (**Johari Window**)
- Always document assumptions

6. Incident Management (CISM Focus)

Goal

Minimize damage + restore operations + learn.

Lifecycle

1. **Preparation**
2. **Detection**
3. **Triage**
4. **Containment**
5. **Eradication**
6. **Recovery**
7. **Lessons Learned**

Planning (IRP, BCP, DRP)

- **IRP:** how to respond to incidents
- **BCP:** keep business running
- **DRP:** restore systems

👉 Must be **aligned with BIA (RTO/RPO)**

Containment

- Short-term: isolate quickly
- Long-term: stabilize environment

Eradication

- Remove root cause (malware, access)
- Patch or rebuild systems

Investigation

- Build **timeline**
- Identify **root cause + scope**
- Classify severity

Evidence Handling

- Maintain **chain of custody**
- Follow **order of volatility**
- Work on **copies only**
- Verify with **hashing**

Documentation

- Record everything (actions, decisions, timelines)
- Critical for legal + improvement

Post-Incident

- Focus on **improvement, not blame**
- Update:
 - Controls
 - Plans
 - Training

7. Business Continuity & Recovery

Backups (3-2-1 Rule)

- 3 copies
- 2 media types
- 1 offsite

👉 Add:

- Encryption
- Immutability (ransomware defense)
- Regular restore testing

Recovery Sites

- **Hot:** instant, expensive
- **Warm:** moderate setup
- **Cold:** slow, cheap

👉 Choose based on **RTO/RPO vs cost**

8. Pentesting (Planning → Recon)

Core Idea

- Plan first → avoid legal risk + wasted effort

Key Documents

- **NDA:** protect data
- **MSA:** overall contract
- **SOW:** specific engagement
- **RoE:** what's allowed
- **Communication Plan:** escalation paths

👉 These prevent **legal issues + scope creep**

Project Management

- Define:
 - Scope
 - Timeline
 - Deliverables
- Take structured notes (CherryTree, OneNote)

Passive Recon (OSINT)

Goal

- Gather intel **without touching target**

What to Find

- Domains, IPs, services
- Tech stack
- Employees/emails
- Leaked credentials
- Public files/metadata

Key Tools/Methods

- **Job postings** → tech stack
- **Shodan** → exposed services
- **Hunter.io** → emails
- **DeHashed** → breaches
- **Google Dorks** → exposed files
- **theHarvester / Amass** → domains
- **MetaGooFil** → document metadata

Why It Matters

- Maps real attack surface
- Finds easy wins (misconfigs, leaks)
- Enables targeted attacks
- Keeps engagement stealthy

9. Final Big Picture (How Everything Connects)

- **Governance** defines rules
- **Risk Management** prioritizes what matters
- **BIA** defines impact (RTO/RPO)
- **Controls** reduce likelihood
- **Incident Management** handles failures
- **BCP/DRP** ensure resilience
- **Pentesting** tests defenses